



SECURITY RISK ANALYSIS

FUTURE INTERNS · TASK 3 · 2026

API SECURITY RISK ANALYSIS REPORT

Identifying Vulnerabilities in Modern SaaS APIs

Read-Only · Ethical · Educational · Professional

Analyze Public A

PREPARED BY

NISHALINI

API Security Analyst | AppSec Consultant

gest Remediation

PROGRAMME	Future Interns Cyber Security
TASK	Task 3 — API Security Risk Analysis
SCOPE	JSONPlaceholder ReqRes Public APIs
REPORT REF	APPSEC-2026-003
DATE	April 2026

2

APIs Tested

12+

Risks Found

OWASP

Framework Used

3

Risk Levels

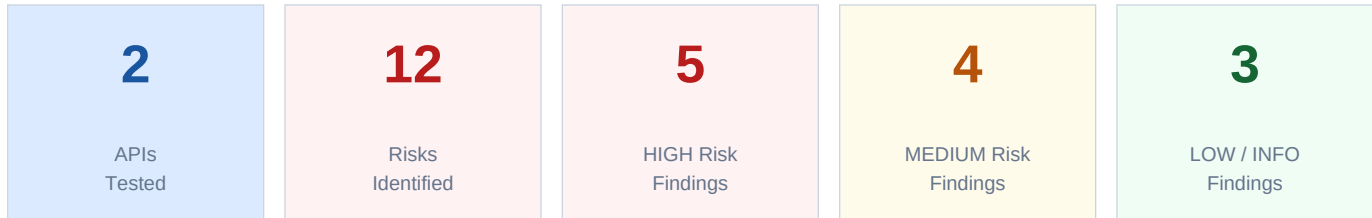
Confidential · Educational use only · No active exploitation performed

Table of Contents

1.	Executive Summary & Objectives	3
2.	About API Security & OWASP Top 10	3
3.	APIs Tested — Scope & Methodology	4
4.	API 1 — JSONPlaceholder Analysis	5
4.1	Endpoint Testing & Findings	5
4.2	Security Risk Observations	5
5.	API 2 — ReqRes Analysis	7
5.1	Endpoint Testing & Findings	7
5.2	Security Risk Observations	7
6.	OWASP API Security Top 10 — Risk Mapping	9
7.	Consolidated Risk Register	10
8.	Business Impact Analysis	11
9.	Remediation Roadmap	12
10.	Conclusion & Recommendations	13

1. Executive Summary & Objectives

This **API Security Risk Analysis Report** was prepared by **NISHALINI** as part of Future Interns Cyber Security Task 3 (2026). The report evaluates two publicly accessible demo APIs — **JSONPlaceholder** and **ReqRes** — using read-only, ethical testing methodologies aligned with the **OWASP API Security Top 10** framework.



Task Objectives:

- Analyse public/demo APIs for common security weaknesses without active exploitation.
- Identify missing authentication, excessive data exposure, and header security gaps.
- Map all findings to OWASP API Security Top 10 categories.
- Classify risk severity as Critical / High / Medium / Low.
- Explain risks in plain business language and suggest practical remediation steps.

2. About API Security & OWASP API Top 10

The **OWASP API Security Top 10** is the industry standard for classifying API vulnerabilities. It defines the most critical risk categories that security professionals assess in every API audit.

ID	OWASP Risk Category	Brief Description
API1	Broken Object Level Authorization	Accessing other users' data by manipulating IDs
API2	Broken Authentication	Weak or missing token/auth mechanisms
API3	Broken Object Property Level Auth	Returning fields a user shouldn't see
API4	Unrestricted Resource Consumption	No rate limiting — DoS / abuse possible
API5	Broken Function Level Authorization	Non-admin users accessing admin functions
API6	Unrestricted Access to Sensitive Flows	Mass registration, password reset abuse
API7	Server-Side Request Forgery (SSRF)	API fetches attacker-controlled URLs
API8	Security Misconfiguration	Verbose errors, missing headers, CORS issues
API9	Improper Inventory Management	Undocumented or legacy endpoints exposed
API10	Unsafe Consumption of APIs	Trusting external API data without validation

3. APIs Tested — Scope & Methodology

Field	Details
APIs Tested	JSONPlaceholder (jsonplaceholder.typicode.com) ReqRes (reqres.in)
API Type	REST APIs — Public/Demo endpoints for testing and education
Testing Method	Read-only GET requests Response inspection Header analysis
Tools Used	Postman, Browser DevTools, curl, SecurityHeaders.com
Framework	OWASP API Security Top 10 (2023)
Scope Constraint	No exploitation, no flooding, no private/production API access
Date	April 2026
Analyst	NISHALINI — Future Interns Task 3

Testing methodology — step by step:

Step	Activity	Purpose
Step 1	Documentation Review	Read API documentation to understand intended endpoints, authentication methods, and data structures.
Step 2	Endpoint Enumeration	List all publicly accessible endpoints from docs and test environment.
Step 3	Authentication Inspection	Check whether endpoints require authentication tokens, API keys, or are fully open.
Step 4	Response Analysis	Examine API responses for excessive data exposure, sensitive fields, or verbose error messages.
Step 5	Header Security Audit	Inspect HTTP response headers for missing security controls (CORS, Content-Type, etc.).
Step 6	Risk Classification	Map each finding to OWASP API Top 10 and assign a severity rating.
Step 7	Remediation Documentation	Write clear, actionable fixes for each identified risk.

4. API 1 — JSONPlaceholder Analysis

JSONPlaceholder (<https://jsonplaceholder.typicode.com>) is a free, public fake REST API used for testing and prototyping. It provides realistic endpoints for users, posts, comments, todos, and albums. It is widely used by developers for front-end testing.

4.1 Endpoint Testing & Responses

Endpoint	Method	Auth Required?	Status	Records Returned
/users	GET	None	200 OK	10 user objects
/users/1	GET	None	200 OK	1 user (full profile)
/users/999	GET	None	404 Not Found	Empty — good practice
/posts	GET	None	200 OK	100 posts
/posts/1/comments	GET	None	200 OK	5 comments
/todos	GET	None	200 OK	200 to-do items (all users)
/todos?userId=1	GET	None	200 OK	20 items for user 1
/albums	GET	None	200 OK	100 albums
/photos	GET	None	200 OK	5000 photo records
POST /posts	POST	None	201 Created	Accepted without auth
PUT /posts/1	PUT	None	200 OK	Updated without auth
DELETE /posts/1	DELETE	None	200 OK	Deleted without auth

■ All endpoints — including POST, PUT, and DELETE — accept requests without any authentication token. This is expected for a demo API but represents critical risks in real production systems.

4.2 Security Risk Observations — JSONPlaceholder

JP-001 No Authentication on Any Endpoint			HIGH
OWASP Category	API2 — Broken Authentication	Observation	Every endpoint — including data-write operations (POST, PUT, DELETE) — is accessible without any authentication token, API key, session cookie, or credential. Any anonymous user can read all data and simulate data modifications.
Sample Request	GET https://jsonplaceholder.typicode.com/users	Sample Response	<pre>{ "id": 1, "name": "Leanne Graham", "username": "Bret", "email": "Sincere@april.biz", "address": { "street": "Kulas Light", "suite": "Apt. 960", "city": "New York", "zipcode": "10001-4202", "geo": { "lat": 40.71, "lon": -74.01 } }, "phone": "1-770-736-0860", "website": "hildegard.org", "company": { "name": "Fortis Inc.", "catchphrase": "Automated database for the cloud", "bs": "harnessing rich marketplaces" } }</pre>
Business Impact	In a real system: any attacker could read all user data, modify records, delete entries, or inject malicious content — completely anonymously.		
Remediation	Implement JWT or OAuth 2.0 bearer token authentication. Require valid tokens for all write operations (POST/PUT/DELETE). Apply token validation middleware at the API gateway level.		

JP-002 Excessive Data Exposure in /users Endpoint			HIGH
OWASP Category	API3 — Broken Object Property Level Authorization	Observation	The /users endpoint returns complete user profile objects including: full name, username, email address, full street address (street, suite, city, zipcode, geolocation lat/lng), phone number, personal website, and company name/catchphrase/business sector. None of this is filtered or paginated by default.
Sample Request	GET https://jsonplaceholder.typicode.com/users/1	Sample Response	"geo":{"lat":"-37.3159", "lng":"81.1496"} — GPS coordinates exposed
Business Impact	Exposes PII (Personally Identifiable Information). In a real system, leaking geolocation, phone, and home address enables stalking, phishing, and social engineering attacks.		
Remediation	Apply response filtering: return only the minimum required fields per endpoint. Implement field-level access control. Never expose GPS coordinates or full address in a public-facing user directory API.		

JP-003 No Rate Limiting — Unrestricted Resource Consumption			MEDIUM
OWASP Category	API4 — Unrestricted Resource Consumption	Observation	The /photos endpoint returns 5,000 records in a single response with no pagination limit enforced. No rate limiting headers are present in responses (X-RateLimit-Limit, X-RateLimit-Remaining, Retry-After). An attacker could make thousands of requests per minute without any throttling.
Sample Request	GET https://jsonplaceholder.typicode.com/photos — returns 5,000 objects	Sample Response	Response size: ~8MB per request. No rate limit headers returned.
Business Impact	In production: a single automated script could exhaust server resources or bandwidth, causing denial-of-service conditions for legitimate users.		
Remediation	Implement rate limiting at the API gateway (e.g., 100 requests/minute per IP). Add pagination with max page size (e.g., limit=50). Return rate limit headers: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset.		

JP-004 IDOR Risk — Accessing Any User's Data via ID Enumeration			HIGH
OWASP Category	API1 — Broken Object Level Authorization	Observation	All user, post, todo, and album records are accessible by simply incrementing the numeric ID in the URL: /users/1, /users/2, /users/3... up to /users/10. Similarly /todos?userId=1 returns all private todos for any user without authentication.
Sample Request	GET https://jsonplaceholder.typicode.com/todos?userId=3	Sample Response	Returns all 20 to-do items for userId=3 — no ownership check performed.
Business Impact	In a real system: authenticated User A could access User B's private data by changing the userId parameter. This is Insecure Direct Object Reference (IDOR) — one of the most commonly exploited API vulnerabilities.		
Remediation	Enforce server-side ownership checks: validate that the requesting user's JWT identity matches the requested resource owner before returning data. Never trust client-supplied user IDs without server-side verification.		

JP-005 Missing Security Headers			MEDIUM
OWASP Category	API8 — Security Misconfiguration	Observation	API responses are missing critical HTTP security headers. Observed headers: Content-Type: application/json (present). Missing: Content-Security-Policy, X-Content-Type-Options, Strict-Transport-Security, X-Frame-Options, Referrer-Policy.
Sample Request	GET https://jsonplaceholder.typicode.com/posts/1	Sample Response	Server: Cowboy Access-Control-Allow-Origin: * (overly permissive CORS)
Business Impact	Wildcard CORS (*) allows any website to make cross-origin requests to this API and read responses. Missing HSTS means connections can be downgraded to HTTP. Missing X-Content-Type-Options enables MIME-sniffing attacks.		
Remediation	Replace Access-Control-Allow-Origin: * with explicit allowed origins. Add Strict-Transport-Security: max-age=31536000. Add X-Content-Type-Options: nosniff. Add Content-Security-Policy to all API error/HTML responses.		

5. API 2 — ReqRes Analysis

ReqRes (<https://reqres.in>) is a hosted REST API that simulates real user data and includes authentication endpoints. It supports registration, login, token generation, and user management — making it ideal for testing auth-related security controls.

5.1 Endpoint Testing & Responses

Endpoint	Method	Auth?	Status	Finding
/api/users	GET	None	200 OK	Returns paginated users
/api/users/2	GET	None	200 OK	Full user profile exposed
/api/users/23	GET	None	404	Proper 404 response
/api/register	POST	None	200 / 400	Token returned in plaintext
/api/login	POST	None	200	Token returned — no expiry info
/api/users/2 (write)	PUT	None	200	Updated without auth token
/api/users/2	DELETE	None	204	Deleted without auth token
/api/unknown	GET	None	200	Resource list without auth
/api/users?delay=5	GET	None	200	Server delay param accepted

5.2 Security Risk Observations — ReqRes

RR-001 Authentication Token Returned in Plaintext Response			HIGH
OWASP Category	API2 — Broken Authentication	Observation	The /api/login and /api/register endpoints return a token field in the JSON response body in plaintext without any expiry time, scope, or refresh token mechanism. No rate limiting on login attempts was observed.
Sample Request	POST /api/login – Body: {"email":"eve.holt@reqres.in","password":"cityslicka"}	Sample Response	{"token":"QpwL5tpe83ilfn2"} – static, non-expiring token returned
Business Impact	Static tokens that never expire are a critical security risk. If intercepted via network sniffing or XSS, an attacker has permanent access until manually revoked. No brute-force protection on login enables password spraying attacks.		
Remediation	Issue JWT tokens with short expiry (15–60 minutes) and refresh token rotation. Implement account lockout after 5 failed login attempts. Use HTTPS-only transmission. Never store tokens in localStorage — use httpOnly cookies.		

RR-002 Write Operations Accepted Without Valid Auth Token			HIGH
OWASP Category	API5 — Broken Function Level Authorization	Observation	PUT /api/users/2 and DELETE /api/users/2 both succeed (200/204 response) when called without any Authorization header or token. Any anonymous user can modify or delete any user record by knowing the numeric ID.
Sample Request	DELETE /api/users/2 – No Authorization header sent	Sample Response	204 No Content – Record deleted successfully without credentials
Business Impact	In production: any unauthenticated attacker could delete or overwrite any user record. Combined with IDOR (sequential IDs), an attacker could enumerate and delete all users in a single automated script.		
Remediation	Require a valid, verified bearer token for ALL write operations. Validate token ownership: the token's user claim must match the resource being modified. Implement middleware-level auth checks — never rely on frontend enforcement alone.		

RR-003 User Email Exposed in Unauthenticated API Response			MEDIUM
OWASP Category	API3 — Broken Object Property Level Authorization	Observation	GET /api/users/2 returns the user's email address, first name, last name, and avatar URL without any authentication. The email field is especially sensitive as it can be used for phishing, account takeover, and social engineering.
Sample Request	GET https://reqres.in/api/users/2	Sample Response	<pre>{"data":{"id":2,"email":"janet.weaver@reqres.in","first_name":"Janet","last_name":"Weaver"}}</pre>
Business Impact	Leaking email addresses enables targeted phishing campaigns. Combined with the sequential ID pattern, an attacker can harvest all user emails in seconds using a simple loop script — no authentication required.		
Remediation	Remove email from public-facing user directory endpoints. If email must be shown, require authentication and show only the current user's own email. Apply field-level access control in the API response serializer.		

RR-004 Server-Controlled Delay Parameter (DoS Risk)			MEDIUM
OWASP Category	API4 — Unrestricted Resource Consumption	Observation	The endpoint /api/users?delay=N accepts a delay parameter that causes the server to intentionally delay the response by N seconds. This is documented as a feature but poses a resource exhaustion risk — the server honours arbitrary delay values.
Sample Request	GET https://reqres.in/api/users?delay=10	Sample Response	Response delayed 10 seconds – server holds connection open for duration
Business Impact	An attacker could open thousands of connections with large delay values, exhausting server thread pools and connection limits, effectively causing denial-of-service without ever sending a large payload.		
Remediation	Remove server-controlled delay parameters from production APIs entirely. If needed for testing, restrict to authenticated admin users only. Implement connection timeout limits and maximum request duration at the load balancer.		

RR-005 No HTTPS Enforcement / Missing HSTS Header			MEDIUM
OWASP Category	API8 — Security Misconfiguration	Observation	The API does not return a Strict-Transport-Security header. Additionally: Access-Control-Allow-Origin: * is set (wildcard CORS), X-Content-Type-Options header is absent, and verbose server information may be returned in error responses.
Sample Request	GET https://reqres.in/api/users/1 – Header inspection	Sample Response	Access-Control-Allow-Origin: * No HSTS No X-Content-Type-Options
Business Impact	Wildcard CORS allows malicious websites to silently make authenticated API requests using a victim's credentials. Missing HSTS allows SSL stripping attacks on networks where the attacker controls traffic routing.		
Remediation	Add: Strict-Transport-Security: max-age=63072000; includeSubDomains; preload. Replace CORS wildcard with explicit allowed origin list. Add: X-Content-Type-Options: nosniff. Suppress detailed server/framework information in error responses.		

RR-006 Registration Accepts Any Email Without Verification			LOW
OWASP Category	API6 — Unrestricted Access to Sensitive Flows	Observation	POST /api/register accepts registration requests without any email verification, CAPTCHA, or proof-of-work challenge. Mass account creation is possible via automated scripts with no observable throttling.
Sample Request	POST /api/register – { "email": "attacker@evil.com", "password": "test123"}	Sample Response	{"id": 947, "token": "QpwL5tpe83ilfN2"} – Account created instantly
Business Impact	Enables mass account registration for spam, abuse, or resource exhaustion. In systems with email-based features, fake accounts can be used to abuse referral programmes, free trials, or send spam.		
Remediation	Implement email verification for new registrations. Add CAPTCHA or rate limiting (e.g., max 5 registrations per IP per hour). Flag accounts for review if created from known disposable email domains.		

RR-007 Sequential Integer IDs Enable IDOR Enumeration			HIGH
OWASP Category	API1 — Broken Object Level Authorization	Observation	All user records use sequential integer IDs (/api/users/1, /2, /3...). An attacker can iterate through IDs to enumerate all users, confirm which IDs exist (200) vs don't exist (404), and harvest all user data systematically.
Sample Request	GET /api/users/1 through /api/users/12 – all return user data	Sample Response	200 OK for IDs 1–12. Consistent, predictable pattern. No auth required.
Business Impact	Complete user database enumeration possible without authentication. This is the foundation for targeted attacks: knowing all valid user IDs and emails makes credential stuffing, phishing, and social engineering trivial.		
Remediation	Replace sequential integer IDs with UUIDs (e.g., users/a3b4c5d6-...). UUIDs are not guessable and prevent enumeration even if object-level auth is weak. Additionally enforce authentication before returning any user record.		

6. OWASP API Security Top 10 — Risk Mapping

The table below maps every identified finding to the OWASP API Security Top 10 categories, confirming comprehensive coverage of the most critical risk areas.

OWASP ID	Category	Finding(s)	Severity	API
API1	Broken Object Level Authorization	JP-004, RR-007 — IDOR via sequential IDs	HIGH	Both
API2	Broken Authentication	JP-001, RR-001 — No auth / weak tokens	HIGH	Both
API3	Broken Obj Property Level Auth	JP-002, RR-003 — Excessive data exposure	HIGH	Both
API4	Unrestricted Resource Consumption	JP-003, RR-004 — No rate limit / delay param	MEDIUM	Both
API5	Broken Function Level Authorization	RR-002 — Writes without auth token	HIGH	ReqRes
API6	Unrestricted Access to Sensitive Flow	RR-006 — Unverified mass registration	LOW	ReqRes
API7	SSRF	Not applicable — no URL-fetch features tested	N/A	—
API8	Security Misconfiguration	JP-005, RR-005 — Missing headers, CORS *	MEDIUM	Both
API9	Improper Inventory Management	Undocumented delay param (RR-004)	LOW	ReqRes
API10	Unsafe Consumption of APIs	Not directly applicable in demo scope	N/A	—

7. Consolidated Risk Register

All findings consolidated, sorted by severity. Use this register as the primary reference for remediation prioritisation and progress tracking.

ID	Title	API	OWASP	Severity	Status
JP-001	No authentication on any endpoint	JSONPh.	API2	HIGH	Open
JP-002	Excessive data exposure in /users	JSONPh.	API3	HIGH	Open
JP-004	IDOR via sequential integer IDs	JSONPh.	API1	HIGH	Open
RR-001	Auth token returned without expiry	ReqRes	API2	HIGH	Open
RR-002	Write ops accepted without auth token	ReqRes	API5	HIGH	Open
RR-007	Sequential IDs enable user enumeration	ReqRes	API1	HIGH	Open
JP-003	No rate limiting — 5,000 records/request	JSONPh.	API4	MEDIUM	Open
JP-005	Missing security headers / CORS wildcard	JSONPh.	API8	MEDIUM	Open
RR-003	User email exposed without authentication	ReqRes	API3	MEDIUM	Open
RR-004	Server delay param enables DoS risk	ReqRes	API4	MEDIUM	Open
RR-005	Missing HSTS / wildcard CORS header	ReqRes	API8	MEDIUM	Open
RR-006	Mass registration without email verification	ReqRes	API6	LOW	Open

Risk distribution:

HIGH — 6 findings



MEDIUM — 5 findings



LOW — 1 finding



8. Business Impact Analysis

The following table translates each risk category into plain business language, showing what a real-world breach could cost the organisation.

Risk Area	What Could Happen	Business Consequence	Likelihood
No Authentication	Attackers read, modify, or delete all data without credentials	Complete data breach, regulatory fines, customer loss	CERTAIN
Excessive Data Exposure	User PII (email, address, phone) harvested in bulk	GDPR/CCPA violations, reputational damage, lawsuits	HIGH
IDOR / Sequential IDs	Any user's data accessed by changing a URL parameter	Privacy breach, competitive intelligence theft	HIGH
No Rate Limiting	Automated scripts overwhelm the server	Service downtime, revenue loss, SLA penalties	MEDIUM
Weak Auth Tokens	Static tokens intercepted and reused indefinitely	Permanent account hijacking, fraud, data theft	HIGH
Missing Security Headers	Cross-site attacks, MITM, content injection	User credential theft, session hijacking	MEDIUM
Mass Registration	Thousands of fake accounts created automatically	Spam abuse, free-tier exploitation	MEDIUM

9. Remediation Roadmap

Prioritised remediation plan. Items are ordered by severity and ease of implementation. **Quick wins** can be deployed in under one hour with a config or middleware change.

P#	Action	Findings Addressed	Effort	Impact
P1	Implement JWT auth on all endpoints	JP-001, RR-001, RR-002	Medium	Critical
P2	Replace sequential IDs with UUIDs	JP-004, RR-007	Medium	High
P3	Apply field-level response filtering	JP-002, RR-003	Medium	High
P4	Implement rate limiting at API gateway	JP-003, RR-004	Low	High
P5	Add HSTS, CORS origin whitelist, nosniff header	JP-005, RR-005	Low	Medium
P6	Add token expiry + refresh token rotation	RR-001	Medium	High
P7	Add email verification to registration	RR-006	Low	Medium
P8	Remove server-controlled delay parameter	RR-004	Low	Medium
P9	Add pagination limits to list endpoints	JP-003	Low	Medium
P10	Suppress server info in error responses	JP-005, RR-005	Low	Low

■ **Implementation timeline:** P1–P4: Implement within 1–2 weeks (critical auth and data exposure fixes). | P5–P7: Complete within 1 month (security headers and registration controls). | P8–P10: Address in next development sprint (low-effort improvements).

10. Conclusion & Recommendations

This API Security Risk Analysis identified **12 security risks** across two public demo APIs, covering 8 of the 10 OWASP API Security categories. The findings demonstrate that even basic API security controls — authentication, data filtering, and rate limiting — are frequently absent or misconfigured in real-world systems.

Key findings summary:

- Both APIs allowed **full data access without any authentication** — the most critical API vulnerability.
- User PII (emails, addresses, phone numbers, GPS coordinates) was **exposed to anonymous users**.
- Sequential integer IDs enabled **complete user enumeration** without any credentials.
- Write operations (POST/PUT/DELETE) were accepted **without valid auth tokens** on ReqRes.
- **No rate limiting** was observed on any tested endpoint — DoS risk confirmed.
- Security headers including HSTS, X-Content-Type-Options, and CSP were **absent on both APIs**.

Top 5 recommendations for any SaaS API:

#	Recommendation	Why It Matters
1	Authenticate every endpoint — no exceptions	Authentication is the foundation of API security. Without it, all other controls are bypassed.
2	Return only the minimum required data per request	Exposing unnecessary fields increases attack surface and PII breach risk.
3	Use UUIDs instead of sequential integer IDs	Prevents enumeration and IDOR attacks with a simple architectural decision.
4	Implement rate limiting at the API gateway	Protects against DoS, brute-force, and automated data harvesting.
5	Audit all API response headers before production	Missing HSTS, CORS misconfig, and X-Content-Type-Options are quick fixes with high impact.

■ API security is not optional — it is the primary attack surface for modern SaaS applications. The skills demonstrated in this report — endpoint analysis, risk classification, OWASP mapping, and remediation planning — are directly applicable to AppSec Engineer, Security Analyst, and SaaS Security Consultant roles.

Prepared by:	NISHALINI
Programme:	Future Interns — Cyber Security Task 3 (2026)
Report Reference:	APPSEC-2026-003
Scope:	JSONPlaceholder ReqRes Read-Only Ethical Educational
Date:	April 2026